

PRAKTIKUM 10

Percoaan 1:

Kode class Queue23:

```
package Pertemuan10.P1Jobsheet10;

public class Queue23 {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue23(int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    boolean isEmpty() {
        return size == 0;
    }

    boolean isFull() {
        return size == max;
    }

    void peek() {
        if (isEmpty()) {
            System.out.println("Queue masih kosong");
            return;
        }
        System.out.println("Elemen terdepan: " + data[front]);
    }

    void print() {
        if (isEmpty()) {
            System.out.println("Queue masih kosong");
            return;
        }
        System.out.println("Isi queue:");
    }
}
```

```

        int i = front;
        for (int j = 0; j < size; j++) {
            System.out.println(data[i]);
            i = (i + 1) % max;
        }
        System.out.println("Jumlah elemen: " + size);
    }

    void clear() {
        if (isEmpty()) {
            System.out.println("Queue sudah kosong");
            return;
        }
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    }

    void enqueue(int dt) {
        if (isFull()) {
            System.out.println("Queue sudah penuh (overflow)!");
            System.out.println("Program dihentikan.");
            System.exit(1);
        }

        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % max;
        }
        data[rear] = dt;
        size++;
    }

    int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue masih kosong (underflow)!");
            System.out.println("Program dihentikan.");
            System.exit(1);
        }

        int dt = data[front];
        size--;
    }

```

```

        if (isEmpty()) {
            front = rear = -1;
        } else {
            front = (front + 1) % max;
        }
        return dt;
    }
}

```

kode di class QueueMain23:

```

package Pertemuan10.P1Jobsheet10;

import java.util.Scanner;

public class QueueMain23 {
    static void menu() {
        System.out.println("\n=== Menu Queue ===");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Peek");
        System.out.println("4. Print");
        System.out.println("5. Clear");
        System.out.println("0. Keluar");
        System.out.print("Pilih: ");
    }

    static int readInt(Scanner sc) {
        while (!sc.hasNextInt()) {
            sc.next();
            System.out.print("Input harus angka. Pilih: ");
        }
        int val = sc.nextInt();
        sc.nextLine();
        return val;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan kapasitas queue: ");
    }
}

```

```

        int n = readInt(sc);
        Queue23 Q = new Queue23(n);

        int pilih;
        do {
            menu();
            pilih = readInt(sc);
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int dataMasuk = readInt(sc);
                    Q.enqueue(dataMasuk);
                    break;
                case 2:
                    int dataKeluar = Q.dequeue();
                    System.out.println("Data yang dikeluarkan: " +
dataKeluar);
                    break;
                case 3:
                    Q.peek();
                    break;
                case 4:
                    Q.print();
                    break;
                case 5:
                    Q.clear();
                    break;
                case 0:
                    System.out.println("Keluar.");
                    break;
                default:
                    System.out.println("Menu tidak tersedia.");
            }
        } while (pilih != 0);

        sc.close();
    }
}

```

Verifikasi Hasil:

```
=== Menu Queue ===
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 1
Masukkan data baru: 10

=== Menu Queue ===
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 2
Data yang dikeluarkan: 10

=== Menu Queue ===
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 1
Masukkan data baru: 23
```

=== Menu Queue ===

1. Enqueue

2. Dequeue

3. Peek

4. Print

5. Clear

0. Keluar

Pilih: 1

Masukkan data baru: 12

=== Menu Queue ===

1. Enqueue

2. Dequeue

3. Peek

4. Print

5. Clear

0. Keluar

Pilih: 1

Masukkan data baru: 45

=== Menu Queue ===

1. Enqueue

2. Dequeue

3. Peek

4. Print

5. Clear

0. Keluar

Pilih: 3

Elemen terdepan: 23

```
PROBLEMS 36 OUTPUT DEBUG CON
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 3
Elemen terdepan: 23

=== Menu Queue ===
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 4
Isi queue:
23
12
45
Jumlah elemen: 3

=== Menu Queue ===
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Clear
0. Keluar
Pilih: 5
Queue berhasil dikosongkan
```

2.1.3 Pertanyaan

1) Pada konstruktor, mengapa front dan rear bernilai -1, sedangkan size bernilai 0?

- front = rear = -1 menandakan queue belum punya elemen (belum ada indeks valid).
- size = 0 menandakan jumlah elemen saat ini memang masih nol.
- Setelah enqueue pertama, barulah front dan rear menjadi 0.

2) Pada method enqueue, maksud/kegunaan potongan kode pengaturan rear dan penyimpanan data

Intinya untuk menambah elemen di belakang (FIFO) dan menjaga indeks tetap valid (circular queue):

- Kalau queue kosong: front = rear = 0
 - Kalau tidak kosong: rear digeser ke indeks berikutnya (circular) lalu data[rear] = dt
- Implementasi yang dipakai ada di Queue23.java (method enqueue).

3) Pada method dequeue, maksud/kegunaan potongan kode pengaturan front dan kondisi saat queue menjadi kosong

Intinya untuk mengeluarkan elemen di depan dan menggeser front:

- Ambil `dt = data[front]`
- Kurangi size
- Jika setelah dikurangi queue kosong: set `front = rear = -1`
- Jika tidak: front digeser ke indeks berikutnya (circular)

Implementasi ada di `Queue23.java` (method `dequeue`).

4) Pada method `print`, mengapa perulangan dimulai dari `i = front`, bukan `i = 0`?

Karena elemen queue yang “aktif” dimulai dari posisi `front`.

Di circular queue, data bisa tersebar, jadi mencetak dari 0 tidak selalu mewakili urutan antrian.

5) Maksud potongan kode pergeseran indeks saat `print` (circular)

Bagian seperti `i = (i + 1) % max` berfungsi agar indeks:

- maju ke elemen berikutnya, dan
- kembali ke 0 jika sudah mencapai `max-1`.

6) Potongan kode yang merupakan queue overflow

Overflow terjadi saat `enqueue` ketika queue penuh:

- `if (isFull()) { ... }` pada `Queue23.java` method `enqueue`.

7) Modifikasi agar overflow/underflow menghentikan program

Modifikasi yang dilakukan:

- File yang dimodifikasi: `Queue23.java`
- Kode yang ditambahkan/diubah: pada kondisi overflow di `enqueue()` dan underflow di `dequeue()` ditambahkan:
 - `System.out.println("Program dihentikan.");`
 - `System.exit(1);`

Percobaan 2:

Kode di class `Mahasiswa23.java`:

```
package Pertemuan10.P2Jobsheet10;

public class Mahasiswa23 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa23(String nim, String nama, String prodi, String
kelas) {
        this.nim = nim;
        this.nama = nama;
```



```

        this.prodi = prodi;
        this.kelas = kelas;
    }

    void tampilkanData() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama   : " + nama);
        System.out.println("Prodi  : " + prodi);
        System.out.println("Kelas : " + kelas);
    }
}

```

Kode di class AntrianLayanan23.java:

```

package Pertemuan10.P2Jobsheet10;

public class AntrianLayanan23 {
    Mahasiswa23[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan23(int n) {
        max = n;
        data = new Mahasiswa23[max];
        size = 0;
        front = rear = -1;
    }

    boolean isEmpty() {
        return size == 0;
    }

    boolean isFull() {
        return size == max;
    }

    void clear() {
        front = rear = -1;
        size = 0;
    }
}

```

```

        System.out.println("Antrian berhasil dikosongkan");
    }

    void enqueue(Mahasiswa23 dt) {
        if (isFull()) {
            System.out.println("Antrian sudah penuh!");
            return;
        }

        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % max;
        }
        data[rear] = dt;
        size++;
    }

    Mahasiswa23 dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian masih kosong!");
            return null;
        }

        Mahasiswa23 dt = data[front];
        data[front] = null;
        size--;

        if (isEmpty()) {
            front = rear = -1;
        } else {
            front = (front + 1) % max;
        }
        return dt;
    }

    void peek() {
        if (isEmpty()) {
            System.out.println("Antrian masih kosong!");
            return;
        }
        System.out.println("Antrian terdepan:");
        data[front].tampilkanData();
    }

```

```

    }

    void print() {
        if (isEmpty()) {
            System.out.println("Antrian masih kosong!");
            return;
        }
        System.out.println("Daftar antrian:");
        int i = front;
        for (int j = 0; j < size; j++) {
            System.out.println("\nAntrian ke-" + (j + 1));
            data[i].tampilkanData();
            i = (i + 1) % max;
        }
    }

    int getJumlahAntrian() {
        return size;
    }

    // Pertanyaan 2.2.3: cek antrian paling belakang
    void lihatAkhir() {
        if (isEmpty()) {
            System.out.println("Antrian masih kosong!");
            return;
        }
        System.out.println("Antrian paling belakang:");
        data[rear].tampilkanData();
    }
}

```

Kode di class LayananAkademikSiakad23.java:

```

package Pertemuan10.P2Jobsheet10;

import java.util.Scanner;

public class LayananAkademikSIKAD23 {
    static int readInt(Scanner sc, String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            sc.next();
        }
    }
}

```

```

        System.out.print(prompt);
    }
    int val = sc.nextInt();
    sc.nextLine();
    return val;
}

static String readLineNonEmpty(Scanner sc, String prompt) {
    String s;
    do {
        System.out.print(prompt);
        s = sc.nextLine().trim();
    } while (s.isEmpty());
    return s;
}

static void menu() {
    System.out.println("\n=== Menu Layanan Akademik (SIAKAD) ===");
    System.out.println("1. Tambah Antrian");
    System.out.println("2. Panggil Antrian");
    System.out.println("3. Lihat Antrian Terdepan");
    System.out.println("4. Lihat Semua Antrian");
    System.out.println("5. Cek Jumlah Antrian");
    System.out.println("6. Cek Antrian Paling Belakang");
    System.out.println("0. Keluar");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    AntrianLayanan23 antrian = new AntrianLayanan23(5);

    int pilihan;
    do {
        menu();
        pilihan = readInt(sc, "Pilih: ");

        switch (pilihan) {
            case 1: {
                String nim = readLineNonEmpty(sc, "NIM : ");
                String nama = readLineNonEmpty(sc, "Nama : ");
                String prodi = readLineNonEmpty(sc, "Prodi : ");
                String kelas = readLineNonEmpty(sc, "Kelas : ");
            }
        }
    }
}

```

```

        antrian.enqueue(new Mahasiswa23(nim, nama, prodi,
kelas));

        break;
    }
    case 2: {
        Mahasiswa23 dipanggil = antrian.dequeue();
        if (dipanggil != null) {
            System.out.println("\nMahasiswa dipanggil:");
            dipanggil.tampilkanData();
        }
        break;
    }
    case 3:
        antrian.peek();
        break;
    case 4:
        antrian.print();
        break;
    case 5:
        System.out.println("Jumlah antrian: " +
antrian.getJumlahAntrian());
        break;
    case 6:
        antrian.lihatAkhir();
        break;
    case 0:
        System.out.println("Keluar.");
        break;
    default:
        System.out.println("Menu tidak tersedia.");
    }
} while (pilihan != 0);

sc.close();
}
}

```

Verifikasi hasil:

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: 1

NIM : 2001

Nama : rozi

Prodi : TI

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: 1

NIM : 2002

Nama : Ozi

Prodi : TI

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: 1

NIM : 2003

Nama : Zi

Prodi : TI

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: 2

Mahasiswa dipanggil:

NIM : 2001

Nama : rozi

Prodi : TI

Kelas : 1G

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: 3

Antrian terdepan:

NIM : 2002

Nama : Ozi

Prodi : TI

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: █

Pilih: 4

Daftar antrian:

Antrian ke-1

NIM : 2002

Nama : Ozi

Prodi : TI

Kelas : 1G

Antrian ke-2

NIM : 2003

Nama : Zi

Prodi : TI

Kelas : 1G

=== Menu Layanan Akademik (SIKAD) ===

1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar

Pilih: █


```

=== Menu Layanan Akademik (SIKAD) ===
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih: 5
Jumlah antrian: 2

```

Jumlah antrian: 2

```

=== Menu Layanan Akademik (SIKAD) ===
1. Tambah Antrian
2. Panggil Antrian
3. Lihat Antrian Terdepan
4. Lihat Semua Antrian
5. Cek Jumlah Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih: 6
Antrian paling belakang:
NIM   : 2003
Nama  : Zi
Prodi : TI
Kelas : 1G

```

2.2.3 Pertanyaan:

method lihatAkhir():

```

void lihatAkhir() {
    if (isEmpty()) {
        System.out.println("Antrian masih kosong!");
        return;
    }
    System.out.println("Antrian paling belakang:");
    data[rear].tampilkanData();
}
}

```

menu opsi 6:

```

        case 6:
            antrian.lihatAkhir();
            break;

```

2.3 TUGAS:

Class Diagram:

MahasiswaKRS23
-String nim -String nama -String prodi -String kelas
+tampilkanData()

AntrianKRS23
-MahasiswaKRS23[] data -int front -int rear -int size -int max
+isEmpty() boolean +isFull() boolean +clear() +enqueue(mhs) +dequeue() MahasiswaKRS23 +printAll() +printFront2() +lihatAkhir() +getJumlahAntrian() int

KRSMain23
-MAX_ANTRIAN = 10 -MAKS_DITANGANI_DPA = 30
+main()

kode di class MahasiswaKRS23:

```
package Pertemuan10.TugasKRS;
```

```

public class MahasiswaKRS23 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public MahasiswaKRS23(String nim, String nama, String prodi, String
kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    void tampilkanData() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama   : " + nama);
        System.out.println("Prodi  : " + prodi);
        System.out.println("Kelas : " + kelas);
    }
}

```

Kode di class AntrianKRS23.java:

```

package Pertemuan10.TugasKRS;

public class AntrianKRS23 {
    MahasiswaKRS23[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianKRS23(int n) {
        max = n;
        data = new MahasiswaKRS23[max];
        size = 0;
        front = rear = -1;
    }

    boolean isEmpty() {
        return size == 0;
    }
}

```

```

}

boolean isFull() {
    return size == max;
}

void clear() {
    front = rear = -1;
    size = 0;
    System.out.println("Antrian berhasil dikosongkan.");
}

void enqueue(MahasiswaKRS23 mhs) {
    if (isFull()) {
        System.out.println("Antrian penuh! Tidak bisa menambah.");
        return;
    }

    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % max;
    }
    data[rear] = mhs;
    size++;
    System.out.println("Mahasiswa berhasil masuk antrian.");
}

MahasiswaKRS23 dequeue() {
    if (isEmpty()) {
        return null;
    }

    MahasiswaKRS23 mhs = data[front];
    data[front] = null;
    size--;

    if (isEmpty()) {
        front = rear = -1;
    } else {
        front = (front + 1) % max;
    }
    return mhs;
}

```

```

    }

    void printAll() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Daftar antrian KRS:");
        int i = front;
        for (int j = 0; j < size; j++) {
            System.out.println("\nAntrian ke-" + (j + 1));
            data[i].tampilkanData();
            i = (i + 1) % max;
        }
    }

    void printFront2() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("2 antrian terdepan:");
        int count = Math.min(2, size);
        int i = front;
        for (int j = 0; j < count; j++) {
            System.out.println("\nAntrian ke-" + (j + 1));
            data[i].tampilkanData();
            i = (i + 1) % max;
        }
    }

    void lihatAkhir() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Antrian paling akhir:");
        data[rear].tampilkanData();
    }

    int getJumlahAntrian() {
        return size;
    }

```

```
}
```

Kode di class KRSMain23.java:

```
package Pertemuan10.TugasKRS;

import java.util.Scanner;

public class KRSMain23 {
    static final int MAX_ANTRIAN = 10;
    static final int MAKS_DITANGANI_DPA = 30;

    static int readInt(Scanner sc, String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            sc.next();
            System.out.print(prompt);
        }
        int val = sc.nextInt();
        sc.nextLine();
        return val;
    }

    static String readLineNonEmpty(Scanner sc, String prompt) {
        String s;
        do {
            System.out.print(prompt);
            s = sc.nextLine().trim();
        } while (s.isEmpty());
        return s;
    }

    static void menu() {
        System.out.println("\n=== Menu Antrian Persetujuan KRS (DPA) ===");
        System.out.println("1. Tambah Antrian (Daftar Mahasiswa)");
        System.out.println("2. Panggil Antrian untuk Proses KRS (2 mahasiswa)");
        System.out.println("3. Tampilkan Semua Antrian");
        System.out.println("4. Tampilkan 2 Antrian Terdepan");
        System.out.println("5. Tampilkan Antrian Paling Akhir");
        System.out.println("6. Cek Antrian Kosong");
    }
}
```

```

        System.out.println("7. Cek Antrian Penuh");
        System.out.println("8. Kosongkan Antrian");
        System.out.println("9. Cetak Jumlah Antrian");
        System.out.println("10. Cetak Jumlah yang Sudah Proses KRS");
        System.out.println("11. Cetak Jumlah yang Belum Proses KRS
(dari kuota 30)");
        System.out.println("0. Keluar");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianKRS23 antrian = new AntrianKRS23(MAX_ANTRIAN);
        int sudahProses = 0;

        int pilihan;
        do {
            menu();
            pilihan = readInt(sc, "Pilih: ");

            switch (pilihan) {
                case 1: {
                    if (sudahProses >= MAKS_DITANGANI_DPA) {
                        System.out.println("Kuota DPA (30 mahasiswa)
sudah terpenuhi. Tidak bisa menerima antrian baru.");
                        break;
                    }
                    String nim = readLineNonEmpty(sc, "NIM : ");
                    String nama = readLineNonEmpty(sc, "Nama : ");
                    String prodi = readLineNonEmpty(sc, "Prodi : ");
                    String kelas = readLineNonEmpty(sc, "Kelas : ");
                    antrian.enqueue(new MahasiswaKRS23(nim, nama,
prodi, kelas));
                    break;
                }
                case 2: {
                    if (sudahProses >= MAKS_DITANGANI_DPA) {
                        System.out.println("Kuota DPA (30 mahasiswa)
sudah terpenuhi. Proses KRS dihentikan.");
                        break;
                    }
                    int diprosesSekarang = 0;
                    for (int i = 0; i < 2; i++) {
                        if (sudahProses >= MAKS_DITANGANI_DPA) {

```

```

        break;
    }
    MahasiswaKRS23 mhs = antrian.dequeue();
    if (mhs == null) {
        break;
    }
    diprosesSekarang++;
    sudahProses++;
    System.out.println("\nDipanggil untuk proses
KRS:");

    mhs.tampilkanData();
}
if (diprosesSekarang == 0) {
    System.out.println("Antrian kosong. Tidak ada
yang diproses.");
} else {
    System.out.println("\nJumlah diproses pada
panggilan ini: " + diprosesSekarang);
}
break;
}
case 3:
    antrian.printAll();
    break;
case 4:
    antrian.printFront2();
    break;
case 5:
    antrian.lihatAkhir();
    break;
case 6:
    System.out.println("Antrian kosong: " +
antrian.isEmpty());
    break;
case 7:
    System.out.println("Antrian penuh: " +
antrian.isFull());
    break;
case 8:
    antrian.clear();
    break;
case 9:

```



```

        System.out.println("Jumlah antrian saat ini: " +
antrian.getJumlahAntrian());
        break;
    case 10:
        System.out.println("Jumlah yang sudah proses KRS: "
+ sudahProses);
        break;
    case 11:
        System.out.println("Jumlah yang belum proses KRS
(dari kuota 30): " + (MAKS_DITANGANI_DPA - sudahProses));
        break;
    case 0:
        System.out.println("Keluar.");
        break;
    default:
        System.out.println("Menu tidak tersedia.");
    }
} while (pilihan != 0);

sc.close();
}
}

```

Verifikasi hasil:

Tambah antrian:

```
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 1
NIM : 101
Nama : rozi
Prodi : TI
Kelas : 1G
Mahasiswa berhasil masuk antrian.
```

```
=== Menu Antrian Persetujuan KRS (DPA) ===
```

```
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
```

2. panggil antrian untuk di proses:

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 2

Dipanggil untuk proses KRS:
NIM   : 101
Nama  : rozi
Prodi : TI
Kelas : 1G

Dipanggil untuk proses KRS:
NIM   : 102
Nama  : Ozi
Prodi : ti
Kelas : 1g

Jumlah diproses pada panggilan ini: 2
```

3. Tampilkan semua antrian yang belum di proses:

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 3
Daftar antrian KRS:

Antrian ke-1
NIM   : 103
Nama  : bozi
Prodi : ti
Kelas : 1g

Antrian ke-2
NIM   : 104
Nama  : cozi
Prodi : ti
Kelas : 1g
```

4. tampilkan 2 antrian terdepan (belum di proses)

```

=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 4
2 antrian terdepan:

Antrian ke-1
NIM   : 103
Nama  : bozi
Prodi : ti
Kelas : 1g

Antrian ke-2
NIM   : 104
Nama  : cozi
Prodi : ti
Kelas : 1g

```

5. Menampilkan antrian paling akhir:

```

=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 5
Antrian paling akhir:
NIM   : 104
Nama  : cozi
Prodi : ti
Kelas : 1g

```

6. cek antrian kosong

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 6
Antrian kosong: false
```

7. cek antrian penuh

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 7
Antrian penuh: false
```

8. kosongkan antrian

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 8
Antrian berhasil dikosongkan.
```

```

=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 6
Antrian kosong: true

```

9. cetak jumlah antrian

```

Antrian kosong: true

=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 9
Jumlah antrian saat ini: 0

```

10. cetak jumlah yang sudah di proses

```

=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 10
Jumlah yang sudah proses KRS: 2

```

11. cetak jmlh yang belum di proses

```
=== Menu Antrian Persetujuan KRS (DPA) ===
1. Tambah Antrian (Daftar Mahasiswa)
2. Panggil Antrian untuk Proses KRS (2 mahasiswa)
3. Tampilkan Semua Antrian
4. Tampilkan 2 Antrian Terdepan
5. Tampilkan Antrian Paling Akhir
6. Cek Antrian Kosong
7. Cek Antrian Penuh
8. Kosongkan Antrian
9. Cetak Jumlah Antrian
10. Cetak Jumlah yang Sudah Proses KRS
11. Cetak Jumlah yang Belum Proses KRS (dari kuota 30)
0. Keluar
Pilih: 11
Jumlah yang belum proses KRS (dari kuota 30): 28
```